CSC-239 *Java Programming*

Chapter 15: Event-Driven Programming (Part 1)

This lecture is being recorded.

November 3, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Announcement: Quiz 3

- Quiz 3 will begin on Monday, November 10.
  - Online (Moodle) Quiz.
  - The Quiz will be open Monday evening (9:00 p.m.) and remain open for approximately one week.
  - Each student will have two hours to complete the Quiz, from the time that they begin.
- The material covered on Quiz 3 will be
  - The lectures of October 1 through October 27.
  - Chapters 8 through 13.
  - The quiz will include True / False, Multiple-Choice questions.

Reminder: Project 2

- The due date for Project 2:

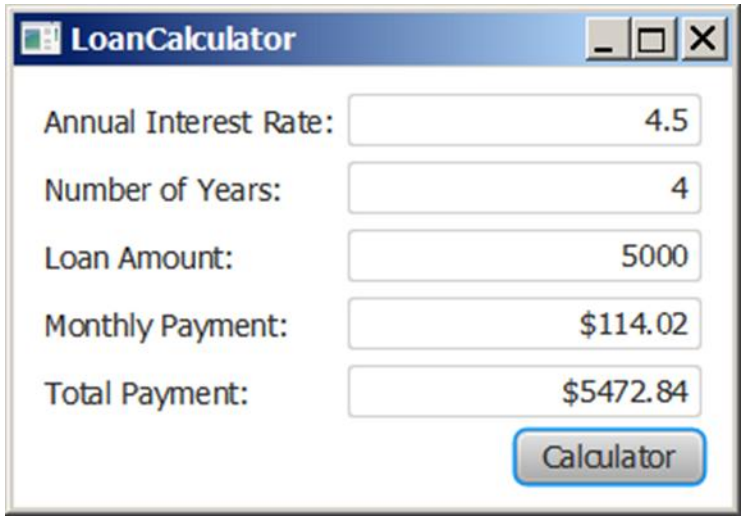
  Monday, November 17

Online Information

- Java Tutorial:
  https://docs.oracle.com/javase/tutorial/
- Java API:
  https://docs.oracle.com/en/java/javase/21/docs/api/index.html
- JavaFX API: https://openjfx.io/javadoc/21/
- The official "*Grader Than*" documentation:
  https://docs.graderthan.com/student/
- *Grader Than* Support E-mail:
  support@graderthan.com
- BHCC Tutoring: https://bhcc.edu/tutoring/

Motivations for Events

- Displaying Graphical objects is of limited use, unless the user can actually interact with the display.
- JavaFX supports the creation of numerous Graphical User Interface (GUI) components:
  - Buttons, that the user can click
  - Text fields, into which the user can type.



Sequential Program Design

- Many simple programs proceed in a sequential manner:
  - Input (either from the user or from a file)
  - Processing (modify data in memory)
  - Output (either to the display, or to a file)
- Character-Menu-driven (interactive) programs:
  - A “main loop” allows user to enter commands in any order.
  - The step-by-step nature is apparent in the source code.

Sequential vs. Asynchronous

- A program with a *Graphical User Interface* (GUI) is structured differently:
  - Many sections of code, that can execute in response to *external* events.
  - Some objects within the program, such as a **Button**, can generate an "event" object.
  - The "event" object gets processed by other objects.
- Students often comment that these programs appear to "jump around" more than the sequential programs that they encountered previously.

Sample Code: `HandleEvent.java`

Similar to JavaFX programs we have seen before.

- Setup and display two buttons:
- Create EventHandler objects:

  This is the code that will actually perform the action.
- Each **Button** object must "register" a particular **EventHandler** object.

  Without this association between the generator and the consumer of the event, the JVM would not know what to do when the user clicks a **Button**.
- But what does the following syntax mean?

```
implements EventHandler<ActionEvent>
```

Remember Interfaces?

- Chapter 13 introduced the concept of an "interface":

```
public interface InterfaceName {
    constant declarations;
    method signatures;
}
```

- If a class claims to "implement" an interface, then the class is "promising" to have the required method(s) within its code.

The EventHandler Interface

When a program includes code such as:

```
class OKHandlerClass implements
                EventHandler<ActionEvent> {
    . . .
}
```

then this program must contain a method such as:

```
public void handle(ActionEvent e) {. . .}
```

Finding the online Documentation

Search the online documents to find the **EventHandler** interface:

1. Use your browser to connect to this URL:

   https://openjfx.io/javadoc/21/

2. Type the desired name into the "Search" field

   (You *may* need to type the ENTER key):



This should open the following web page:

https://openjfx.io/javadoc/21/javafx.base/javafx/event/EventHandler.html

Definition of the **EventHandler** Interface

- When we find the online documentation for **EventHandler**, we see:

```
public interface
    EventHandler<T extends Event>
        extends java.util.EventListener
```

- A class that "implements" the **EventHandler** interface must have a method:

```
void handle(T event){ . . . }
```

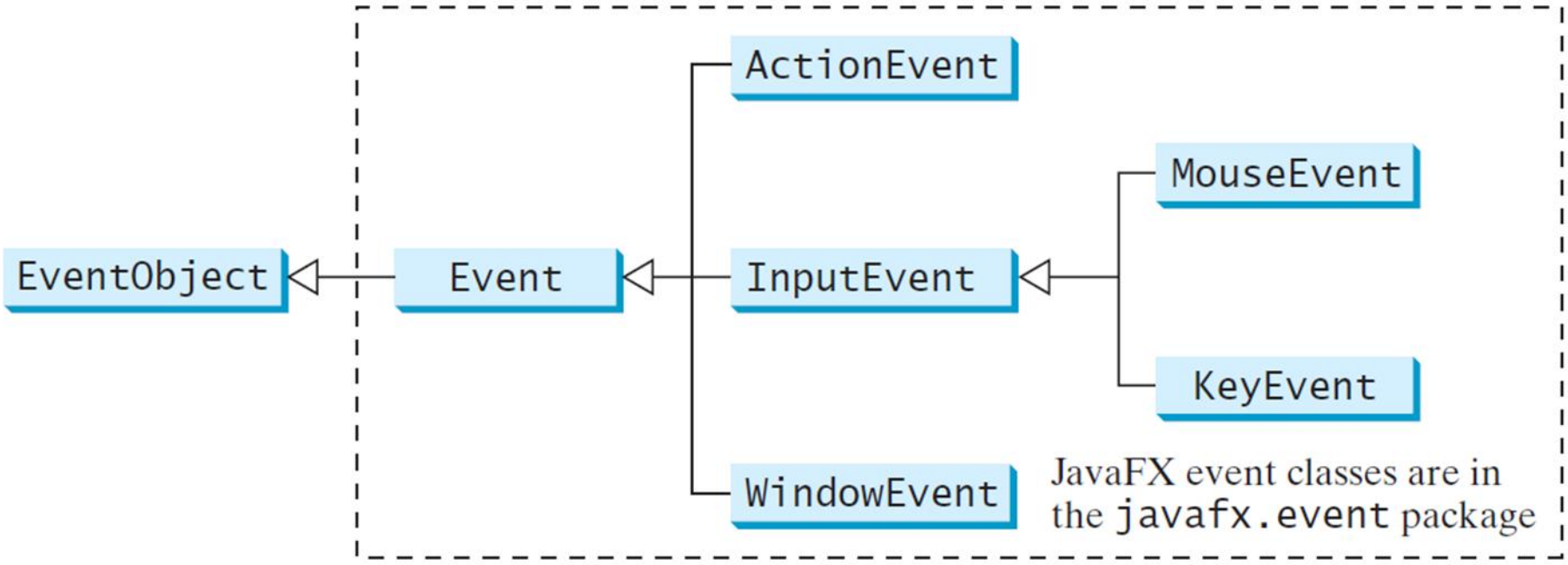
- What is meant by `<T extends Event>` is that the type **`T`** must be a subclass of the **`Event`** class.

Events

- An *event* can be defined as a type of signal to the program that something has happened.
- The event is generated by external user actions such as mouse movements, mouse clicks, or keystrokes.
  - We say that the GUI component, such as a Button, "Fires an event".
  - This generates an **Event** object, causing the appropriate *event handler* code to execute.

Sub-classes of the **Event** Class

The UML diagram below illustrates the inheritance relationship among various types of **Event**.



Event is a subclass of **EventObject**

As with any inheritance hierarchy:

- Shared methods and properties are defined in the *superclass*.
- Specialized features are in the *subclass*.

For example: the **EventObject** class defines a **`getSource()`** method:

  - The **`getSource()`** method returns a reference to the object that generated the event.

EventObject Class

The online documentation for **EventObject** shows:

Method Summary

All Methods | **Instance Methods** | **Concrete Methods**

Modifier and Type	Method	Description
`Object`	`getSource()`	The object on which the Event initially occurred.
`String`	`toString()`	Returns a String representation of this EventObject.

and when we click on **`getSource()`** we see:

getSource

```
public Object getSource()
```

The object on which the Event initially occurred.

Returns:

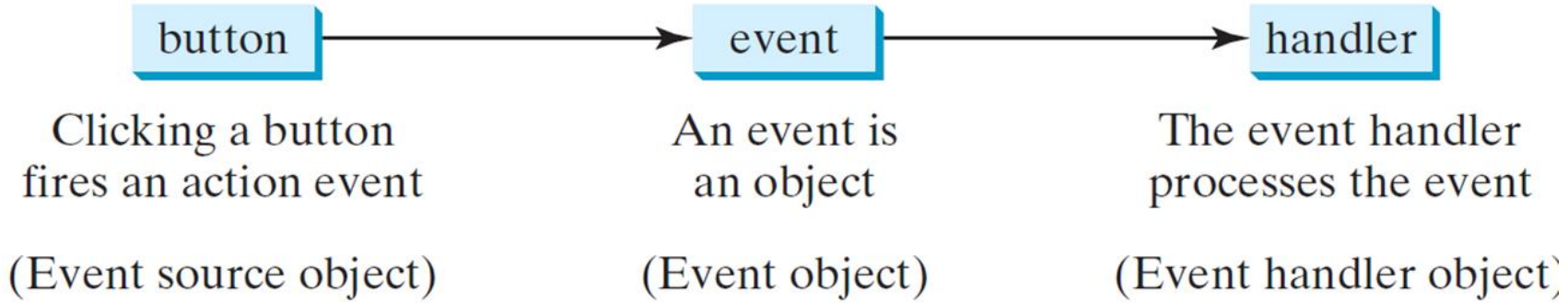
the object on which the Event initially occurred

Another Example: `ControlCircle`

- Creates a **CirclePane** object:
  - subclass of StackPane (Nodes always centered)
  - draws a Circle node
  - has methods to enlarge, shrink the circle.
- Create an **HBox** object:
  - to hold Buttons (arranged in a horizontal row)
- Create a **BorderPane** object:
  - to hold the CirclePane and HBox object.
- Create classes to handle events
- Each **Button** object must "register" a particular **EventHandler** object.
  - Without this association between the generator and the consumer of the event, the JVM would not know what to do when the user clicks a Button.

Handling GUI Events

- Source object (e.g., button)
- Listener object contains a method for processing the event.



Selected User Actions and Handlers

- Table 15.1 (page 597) is a list of information about various events:
  - User Action: "click a button"
  - Source Object: Button
  - Event Type Fired: ActionEvent
  - Event Registration Method: How you link the source object with the EventHandler object.
- An image of Table 15.1 is on the next slide.

Table 15.1 (textbook page 597)

User Action	*Source Object*	*Event Type Fired*	*Event Registration Method*
Click a button	`Button`	`ActionEvent`	`setOnAction(EventHandler<ActionEvent>)`
Press Enter in a text field	`TextField`	`ActionEvent`	`setOnAction(EventHandler<ActionEvent>)`
Check or uncheck	`RadioButton`	`ActionEvent`	`setOnAction(EventHandler<ActionEvent>)`
Check or uncheck	`CheckBox`	`ActionEvent`	`setOnAction(EventHandler<ActionEvent>)`
Select a new item	`ComboBox`	`ActionEvent`	`setOnAction(EventHandler<ActionEvent>)`
Mouse pressed	`Node, Scene`	`MouseEvent`	`setOnMousePressed(EventHandler<MouseEvent>)`
Mouse released			`setOnMouseReleased(EventHandler<MouseEvent>)`
Mouse clicked			`setOnMouseClicked(EventHandler<MouseEvent>)`
Mouse entered			`setOnMouseEntered(EventHandler<MouseEvent>)`
Mouse exited			`setOnMouseExited(EventHandler<MouseEvent>)`
Mouse moved			`setOnMouseMoved(EventHandler<MouseEvent>)`
Mouse dragged			`setOnMouseDragged(EventHandler<MouseEvent>)`
Key pressed	`Node, Scene`	`KeyEvent`	`setOnKeyPressed(EventHandler<KeyEvent>)`
Key released			`setOnKeyReleased(EventHandler<KeyEvent>)`
Key typed			`setOnKeyTyped(EventHandler<KeyEvent>)`

Registering Handlers

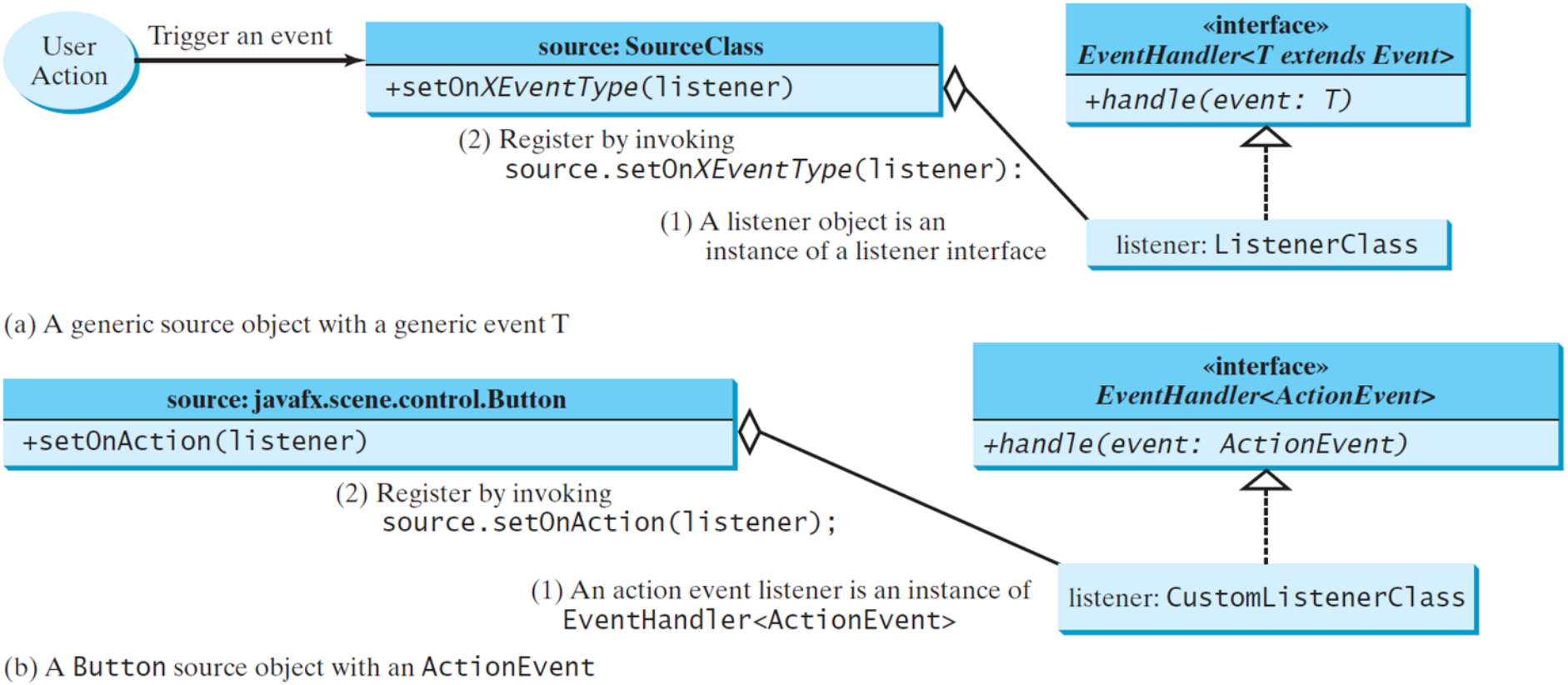
- A source object “fires an event”.
- An “interested object” (the event handler object) processes the event.
  - The handler object must be an instance of the corresponding event-handler interface.

    `EventHandler<T extends Event>`

  - The handler object must be “registered” by the source object.

The Delegation Model

- Figure 15-5 (page 598):



The Delegation Model: Example

(Taken from the **`HandleEvent.java`** example)

- The event-generating (source) object:

```
Button btOK = new Button("OK");
```

- The EventHandler (listener) object:

```
OKHandlerClass handler =
        new OKHandlerClass();
```

- Registration logically links the two objects:

```
btOK.setOnAction(handler);
```

Lab 14a: Beginning JavaFX

- This Lab Exercise is intended to get you started with a very basic JavaFX program.
- The assignment includes a “Starter Code” (sample) program that you can use as a guide for coding other assignments.